



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



DocuMind - Research Assistant Agent

Lab CA-1 AI

Shreya Menon 16010123324

Shreyans Tatiya 16010123325

Siddhant Raut 16010123331

Problem & Motivation

The Problem: Information Overload

- Users deal with large, dense documents (PDFs, research papers, reports)
- Traditional search (Ctrl+F) fails for complex, context-based queries
- Difficult to both **extract insights** and **answer specific questions**

Outcome

- Enables **natural language Q&A** over documents
- Faster, smarter, and more reliable knowledge extraction
- Combines **local + external intelligence** seamlessly

Key Challenges Addressed

Hybrid Retrieval

LLMs lack real-time and private data awareness. ReAct Agent switches between document and web search.



Document Understanding

LLMs struggle with large documents due to context limits. RAG pipeline with semantic chunking is the solution.



Semantic Retrieval

Keyword search fails to capture intent and answer questions. Dense vector embeddings and FAISS retrieval are the solution.

Agent Role

Goal of the Agent

- Bridge the gap between static data (documents) and dynamic user queries
- Transform natural language queries into intelligent search operations
- Return structured insights instead of raw text

Agent Role: AI Research Assistant

- Acts as an autonomous intermediary between user and data
- Dynamically orchestrates tools:
 - document_search
 - summarize_document
 - web_search

Core Responsibilities



Understand Queries

Uses zero-shot ReAct reasoning to interpret complex questions.



Answer Questions

Retrieves semantic matches from FAISS vector database and switches to web search.



Generate Summaries

Compresses large documents into structured markdown insights for fast understanding.

Agent Design - Core Concepts

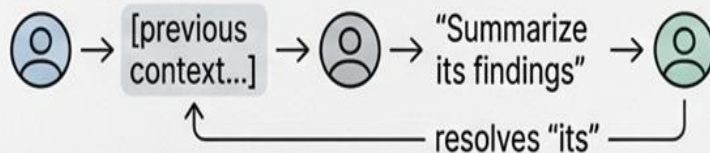
1. Agent Type

 **LangChain: ZERO_SHOT_REACT_DESCRIPTION**

- No hardcoded logic or fine-tuning
- Selects tools dynamically using tool descriptions + query intent

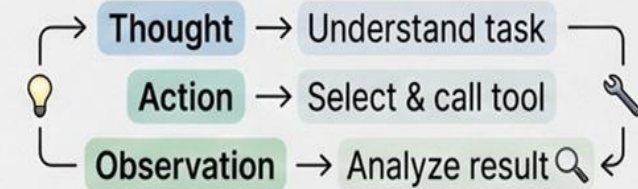
3. Memory (Short-Term Context)

- Maintains conversation history buffer
- Enables contextual understanding of follow-up queries



2. Planning Strategy — ReAct Loop

Follows multi-step reasoning cycle:

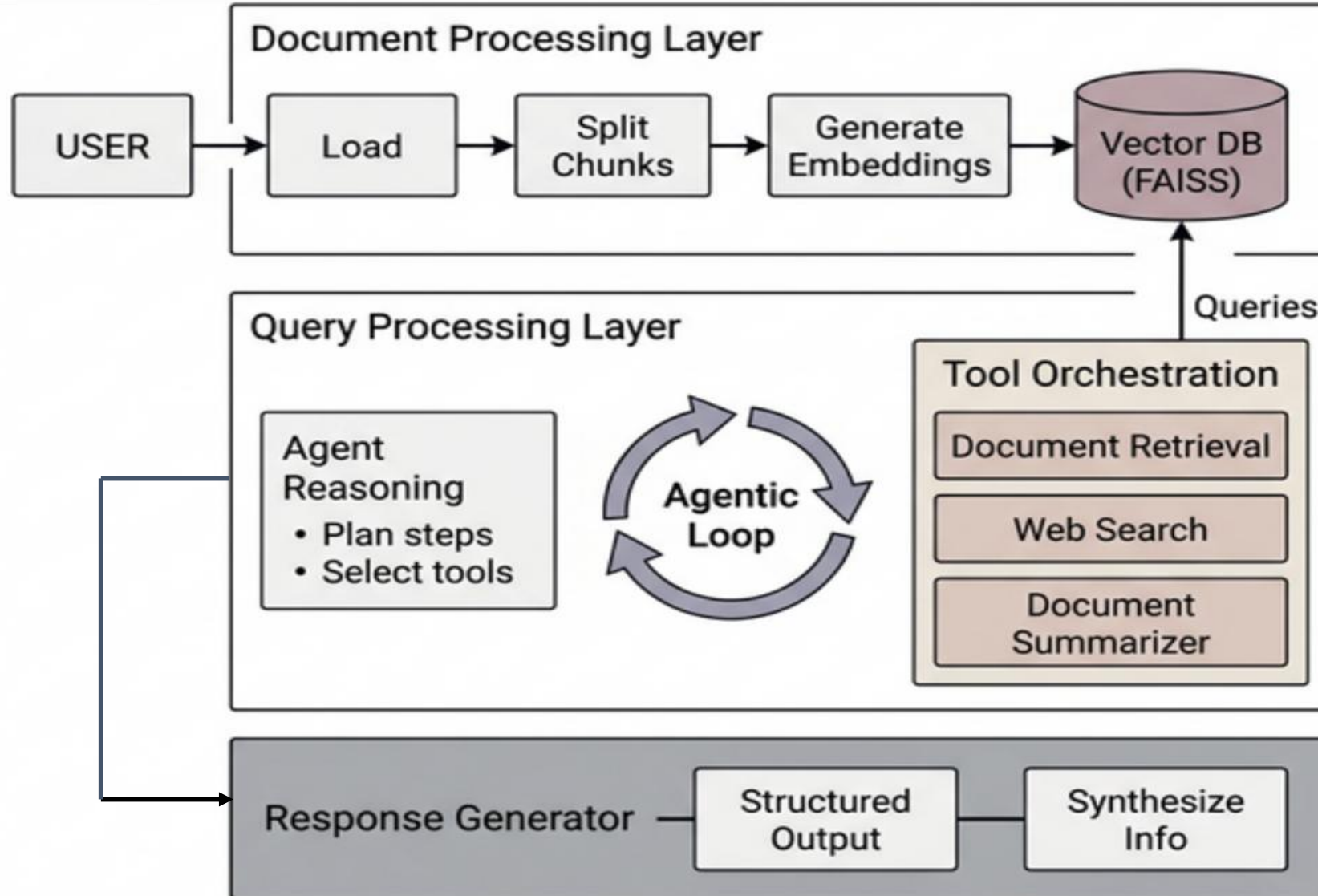


Repeats until final structured response is generated

4. Tools Orchestrated by Agent

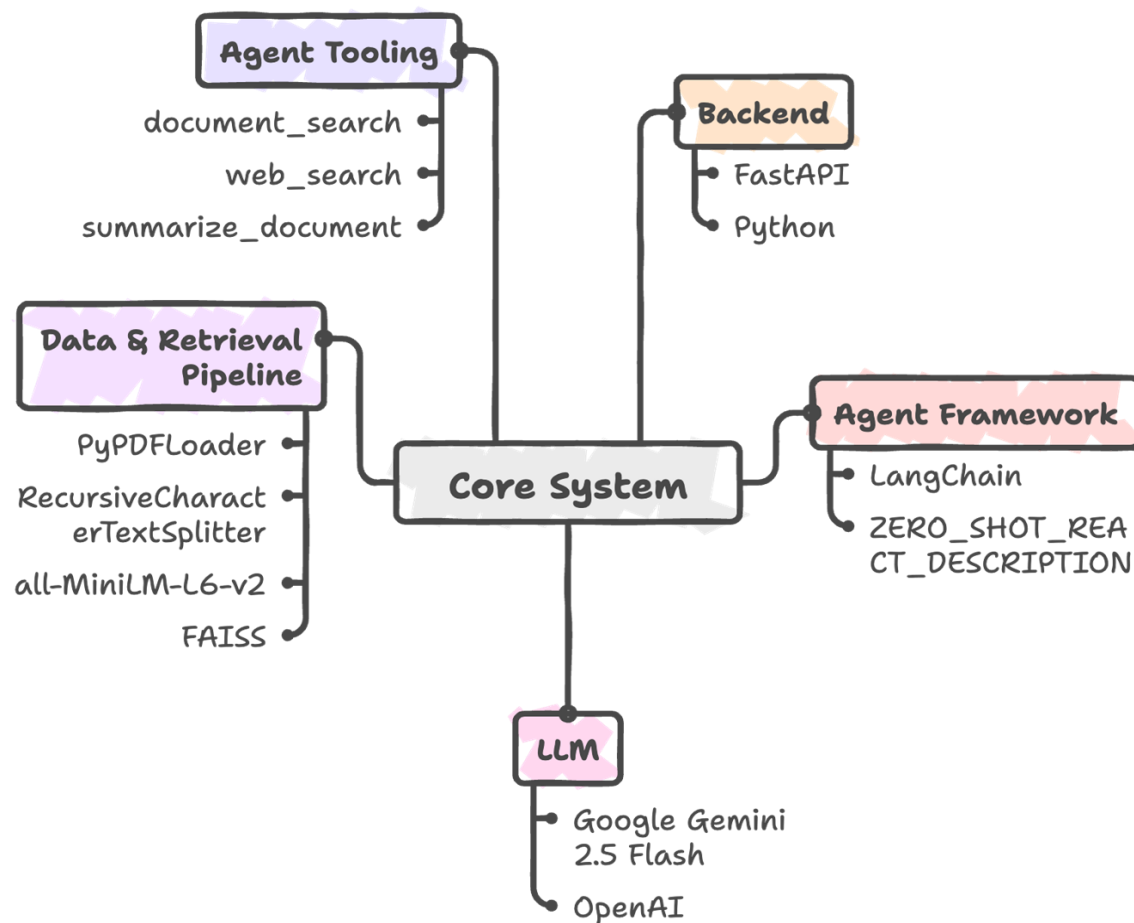
 document_search	→ FAISS + MiniLM for semantic retrieval from PDFs
 web_search	→ DuckDuckGo for real-time external data
 summarize_document	→ Full-document compression for high-level insights

Architecture Diagram

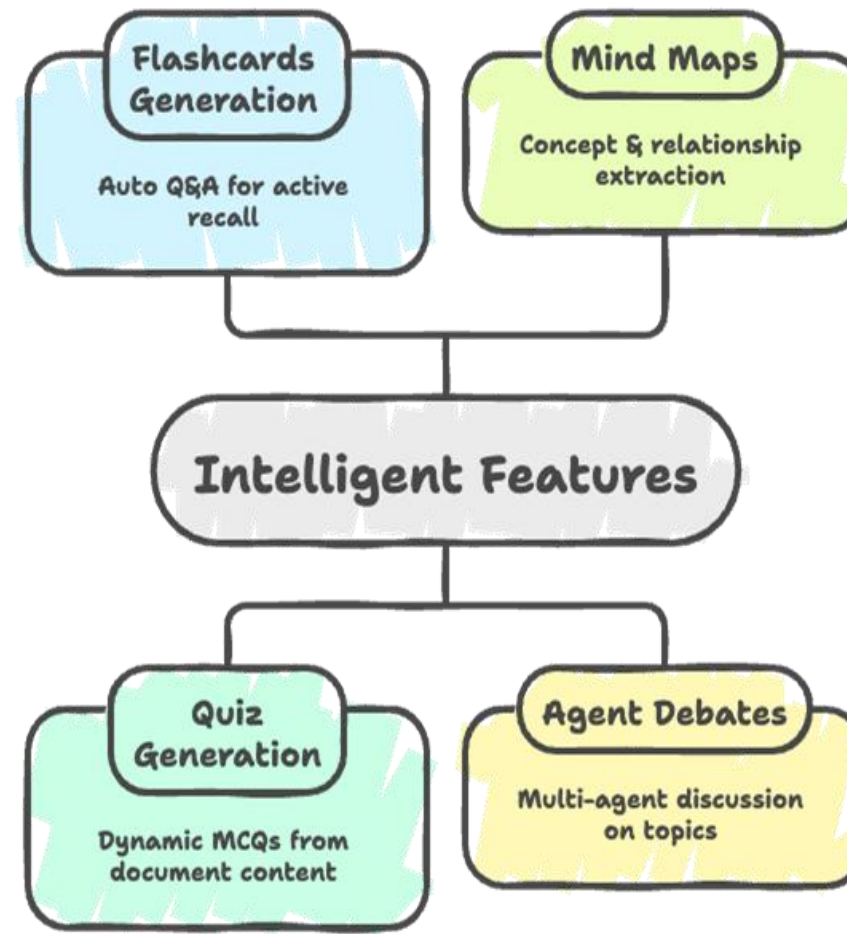


Implementation

Core System Architecture and Components



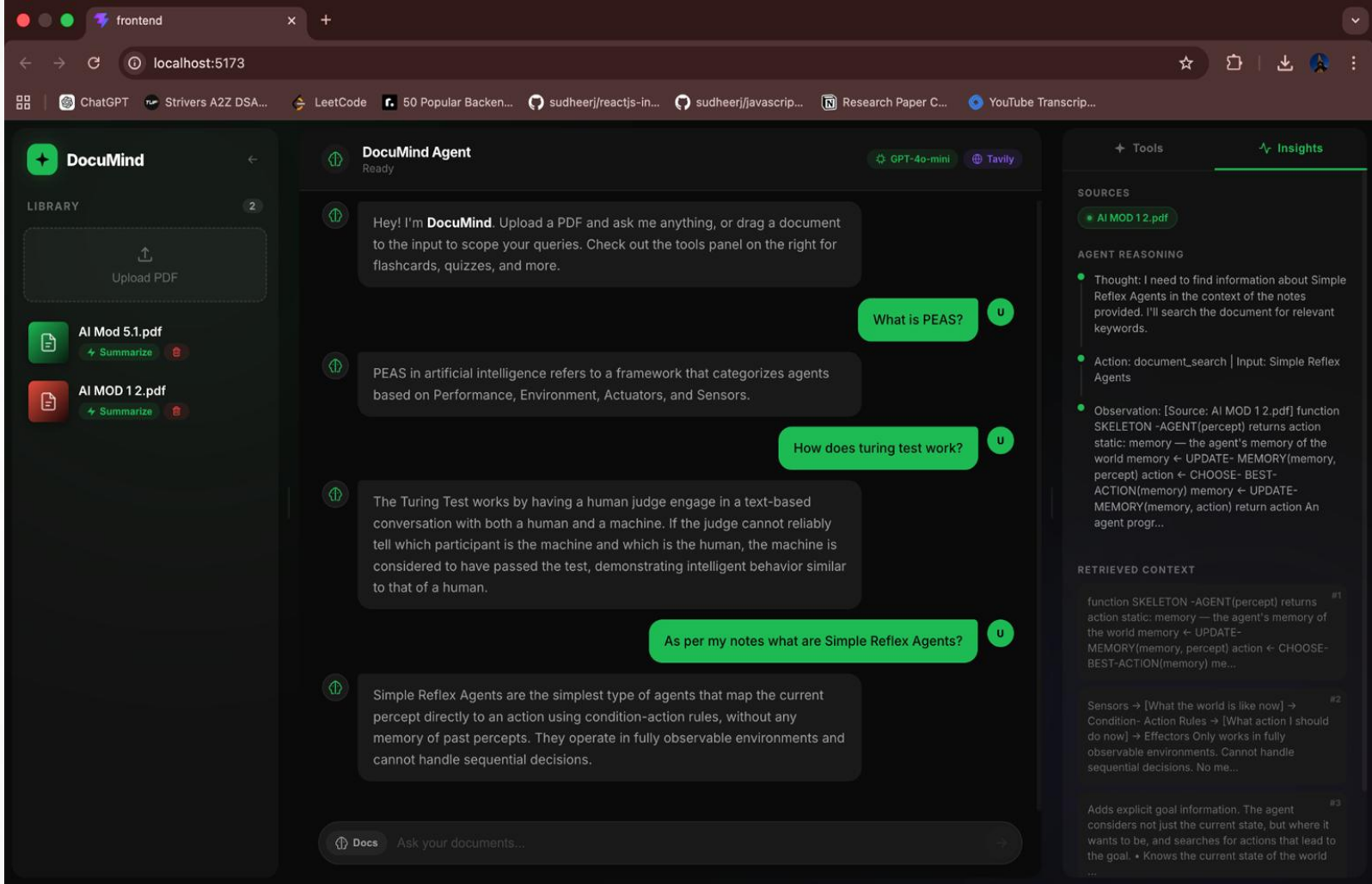
Enhancing Learning with Intelligent Features



Execution with Trace / Reasoning

Document Search

- The agent follows a **ReAct reasoning loop** (Thought → Action → Observation)
- It first identifies the need for **document-based information**
- Selects the **document_search** tool to retrieve relevant content
- Observes retrieved data and generates a **context-aware answer**
- Ensures transparency through **visible reasoning traces and source citations**

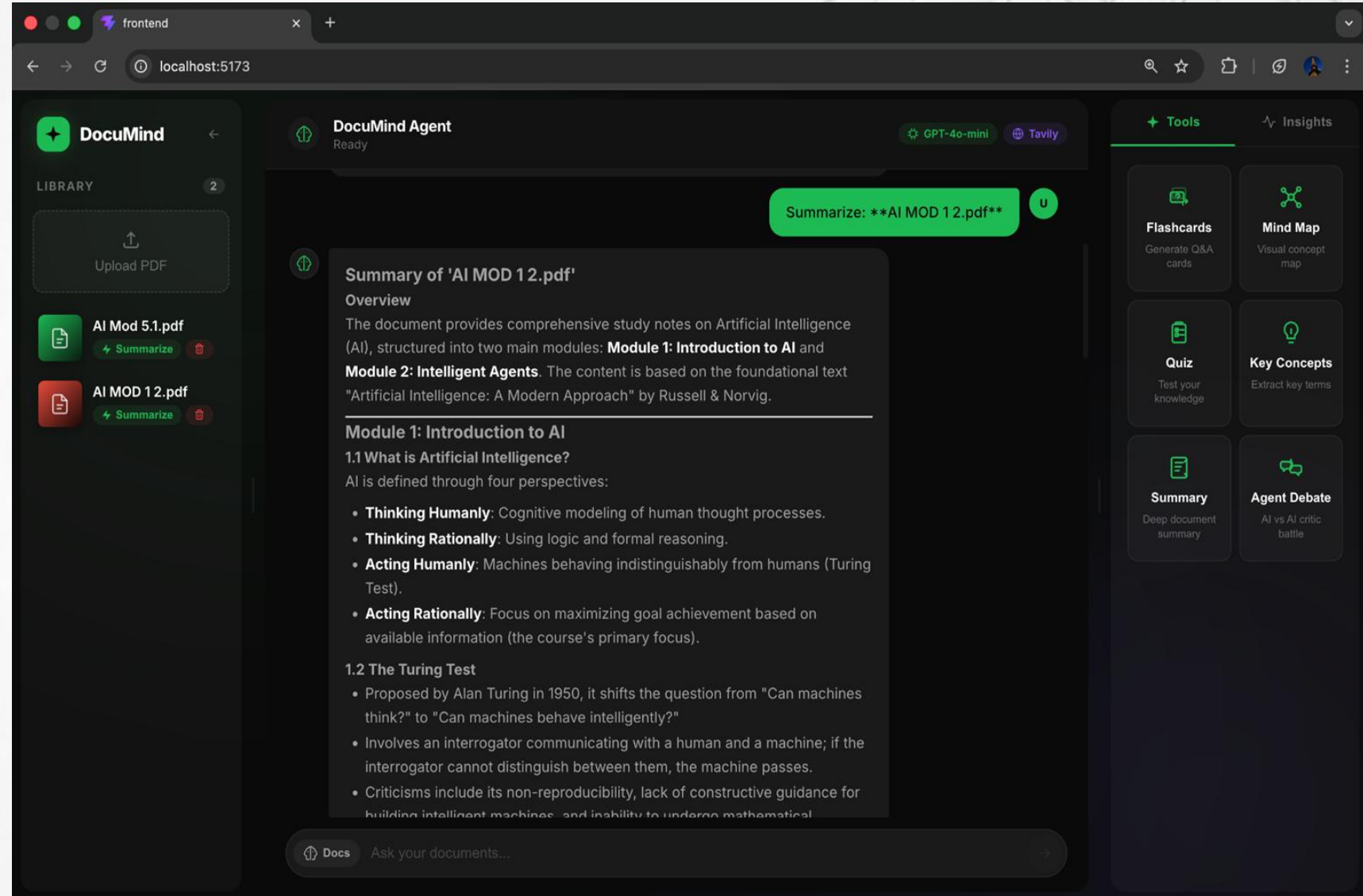


The screenshot displays the DocuMind Agent interface in a web browser. The interface is divided into several sections:

- LIBRARY:** A sidebar on the left showing a list of documents: "AI Mod 5.1.pdf" and "AI MOD 12.pdf". Each document has a "Summarize" button and a red icon.
- DocuMind Agent:** The main chat area where the user interacts with the agent. The agent is "Ready" and uses "GPT-4o-mini" and "Tavily" models. The chat history shows the following interactions:
 - User: "Hey! I'm DocuMind. Upload a PDF and ask me anything, or drag a document to the input to scope your queries. Check out the tools panel on the right for flashcards, quizzes, and more."
 - User: "What is PEAS?"
 - Agent: "PEAS in artificial intelligence refers to a framework that categorizes agents based on Performance, Environment, Actuators, and Sensors."
 - User: "How does turing test work?"
 - Agent: "The Turing Test works by having a human judge engage in a text-based conversation with both a human and a machine. If the judge cannot reliably tell which participant is the machine and which is the human, the machine is considered to have passed the test, demonstrating intelligent behavior similar to that of a human."
 - User: "As per my notes what are Simple Reflex Agents?"
 - Agent: "Simple Reflex Agents are the simplest type of agents that map the current percept directly to an action using condition-action rules, without any memory of past percepts. They operate in fully observable environments and cannot handle sequential decisions."
- Tools:** A panel on the right showing the "document_search" tool selected.
- Insights:** A panel on the right showing the agent's reasoning process:
 - SOURCES:** "AI MOD 12.pdf"
 - AGENT REASONING:**
 - Thought: I need to find information about Simple Reflex Agents in the context of the notes provided. I'll search the document for relevant keywords.
 - Action: document_search | Input: Simple Reflex Agents
 - Observation: [Source: AI MOD 12.pdf] function SKELETON -AGENT(percept) returns action static: memory — the agent's memory of the world memory ← UPDATE- MEMORY(memory, percept) action ← CHOOSE- BEST- ACTION(memory) memory ← UPDATE- MEMORY(memory, action) return action An agent progr...
 - RETRIEVED CONTEXT:**
 - #1: function SKELETON -AGENT(percept) returns action static: memory — the agent's memory of the world memory ← UPDATE- MEMORY(memory, percept) action ← CHOOSE- BEST-ACTION(memory) me...
 - #2: Sensors → [What the world is like now] → Condition- Action Rules → [What action I should do now] → Effectors Only works in fully observable environments. Cannot handle sequential decisions. No me...
 - #3: Adds explicit goal information. The agent considers not just the current state, but where it wants to be, and searches for actions that lead to the goal. • Knows the current state of the world

Summarize

- Generates structured summary from document
- Uses RAG + LLM reasoning
- Triggered via summarize_document tool
- Outputs clear, concise insights

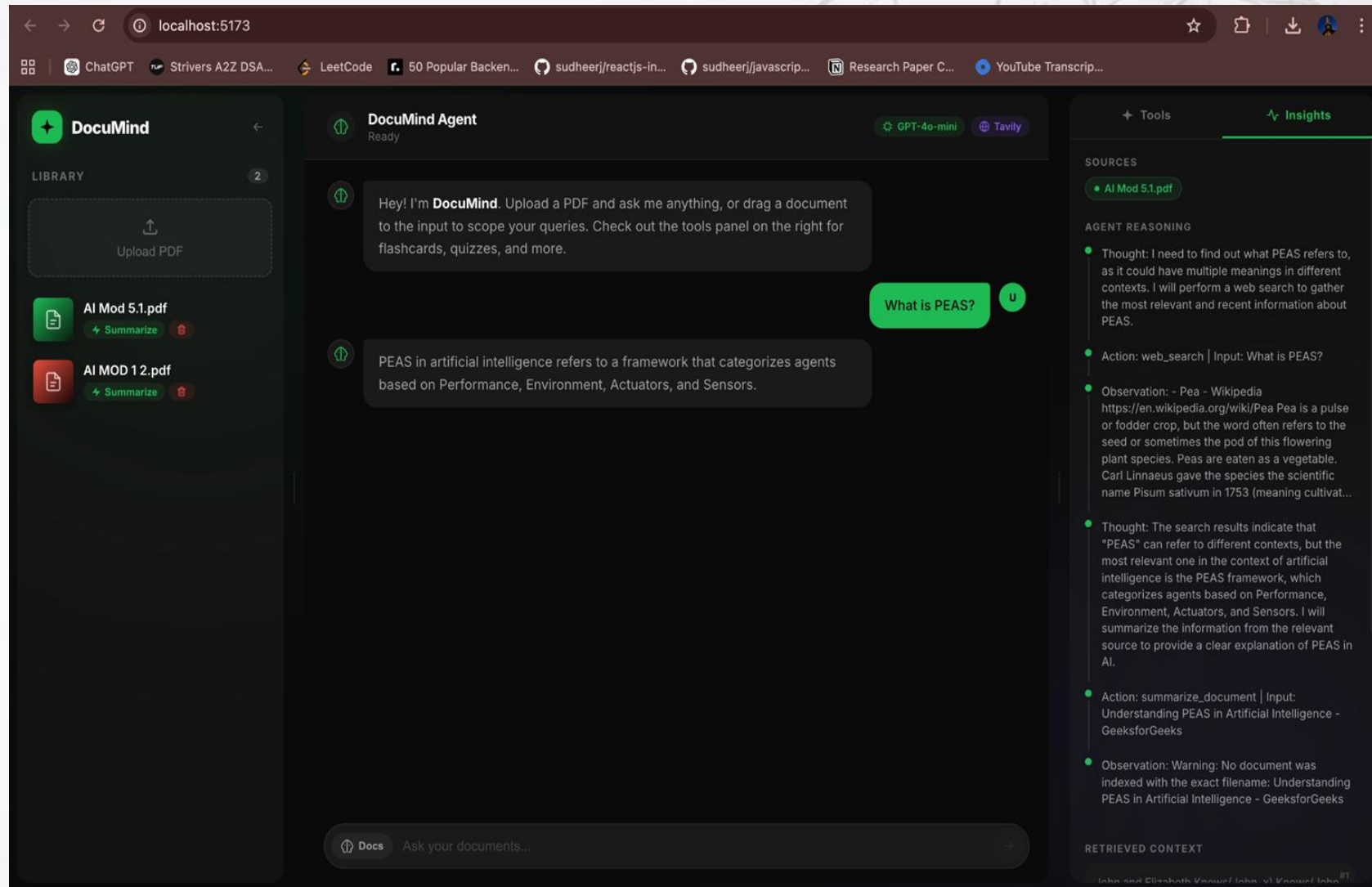


The screenshot displays the DocuMind Agent web interface. On the left, a 'LIBRARY' sidebar shows two PDFs: 'AI Mod 5.1.pdf' and 'AI MOD 12.pdf', each with a 'Summarize' button. The main area features a 'DocuMind Agent' header with 'Ready' status and model options 'GPT-4o-mini' and 'Tavily'. A green button at the top right says 'Summarize: **AI MOD 12.pdf**'. The central content shows a 'Summary of 'AI MOD 12.pdf'' with an 'Overview' section stating the document covers study notes on Artificial Intelligence (AI), structured into two main modules: **Module 1: Introduction to AI** and **Module 2: Intelligent Agents**. The summary is based on the foundational text "Artificial Intelligence: A Modern Approach" by Russell & Norvig. Below this, 'Module 1: Introduction to AI' is detailed, starting with '1.1 What is Artificial Intelligence?' which defines AI through four perspectives: **Thinking Humanly** (Cognitive modeling of human thought processes), **Thinking Rationally** (Using logic and formal reasoning), **Acting Humanly** (Machines behaving indistinguishably from humans (Turing Test)), and **Acting Rationally** (Focus on maximizing goal achievement based on available information (the course's primary focus)). '1.2 The Turing Test' is also outlined, mentioning its proposal by Alan Turing in 1950, the interrogator's role, and criticisms like non-reproducibility and lack of constructive guidance.

On the right, a 'Tools' sidebar offers various functions: 'Flashcards' (Generate Q&A cards), 'Mind Map' (Visual concept map), 'Quiz' (Test your knowledge), 'Key Concepts' (Extract key terms), 'Summary' (Deep document summary), and 'Agent Debate' (AI vs AI critic battle). At the bottom, a 'Docs' section prompts the user to 'Ask your documents...'.

Web Search + Reasoning

- Agent detects missing knowledge
- Uses web_search via ReAct reasoning

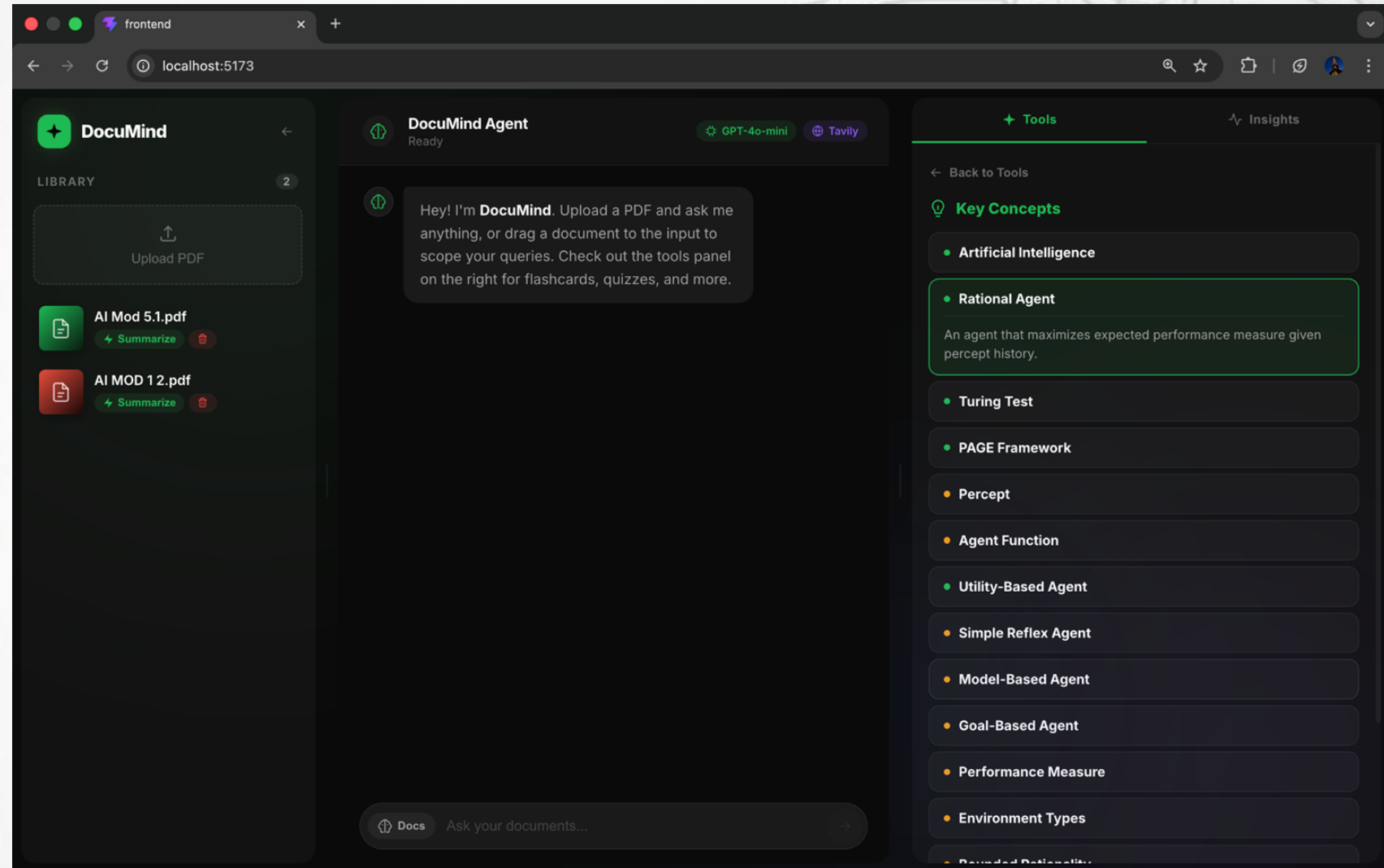


The screenshot displays the DocuMind Agent interface in a web browser. The interface is divided into several sections:

- Left Panel (Library):** Contains a list of documents: "AI Mod 5.1.pdf" and "AI MOD 12.pdf". Each document has a "Summarize" button and a trash icon.
- Top Bar:** Shows the browser address bar at "localhost:5173" and a navigation bar with various application icons.
- DocuMind Agent Chat:** The main chat area shows a conversation. The agent says: "Hey! I'm DocuMind. Upload a PDF and ask me anything, or drag a document to the input to scope your queries. Check out the tools panel on the right for flashcards, quizzes, and more." The user asks: "What is PEAS?". The agent responds: "PEAS in artificial intelligence refers to a framework that categorizes agents based on Performance, Environment, Actuators, and Sensors."
- Right Panel (Tools and Insights):** Contains a "Tools" section with "GPT-4o-mini" and "Tavily". Below it is the "AGENT REASONING" section, which shows the agent's thought process: "Thought: I need to find out what PEAS refers to, as it could have multiple meanings in different contexts. I will perform a web search to gather the most relevant and recent information about PEAS." It then shows the action: "Action: web_search | Input: What is PEAS?". This is followed by an observation from Wikipedia about "Pea" and another thought: "Thought: The search results indicate that 'PEAS' can refer to different contexts, but the most relevant one in the context of artificial intelligence is the PEAS framework, which categorizes agents based on Performance, Environment, Actuators, and Sensors. I will summarize the information from the relevant source to provide a clear explanation of PEAS in AI." The final action is: "Action: summarize_document | Input: Understanding PEAS in Artificial Intelligence - GeeksforGeeks". At the bottom, it shows an observation: "Warning: No document was indexed with the exact filename: Understanding PEAS in Artificial Intelligence - GeeksforGeeks".

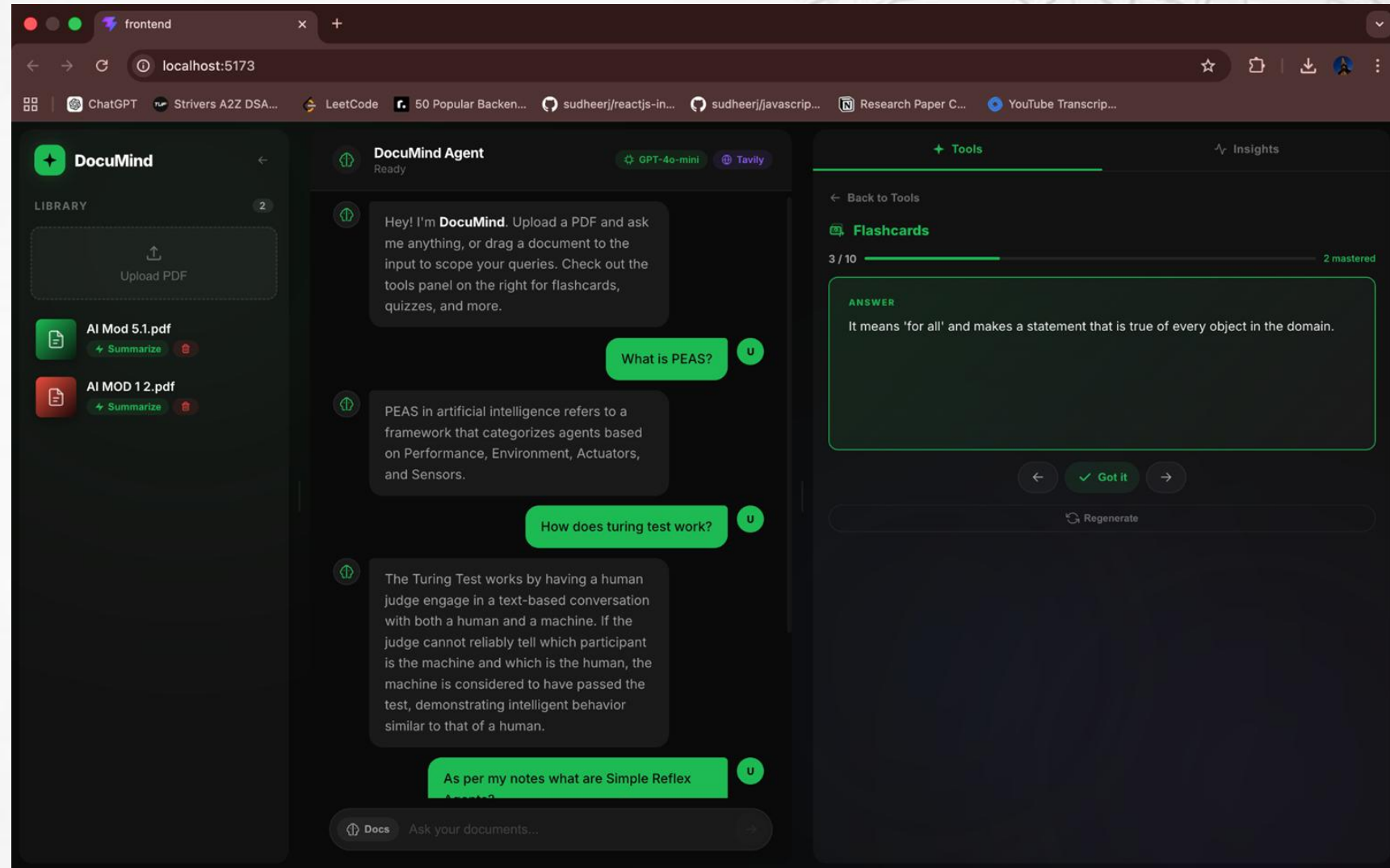
Key Concepts Feature

- Extracts important concepts from documents
- Uses semantic retrieval + LLM processing



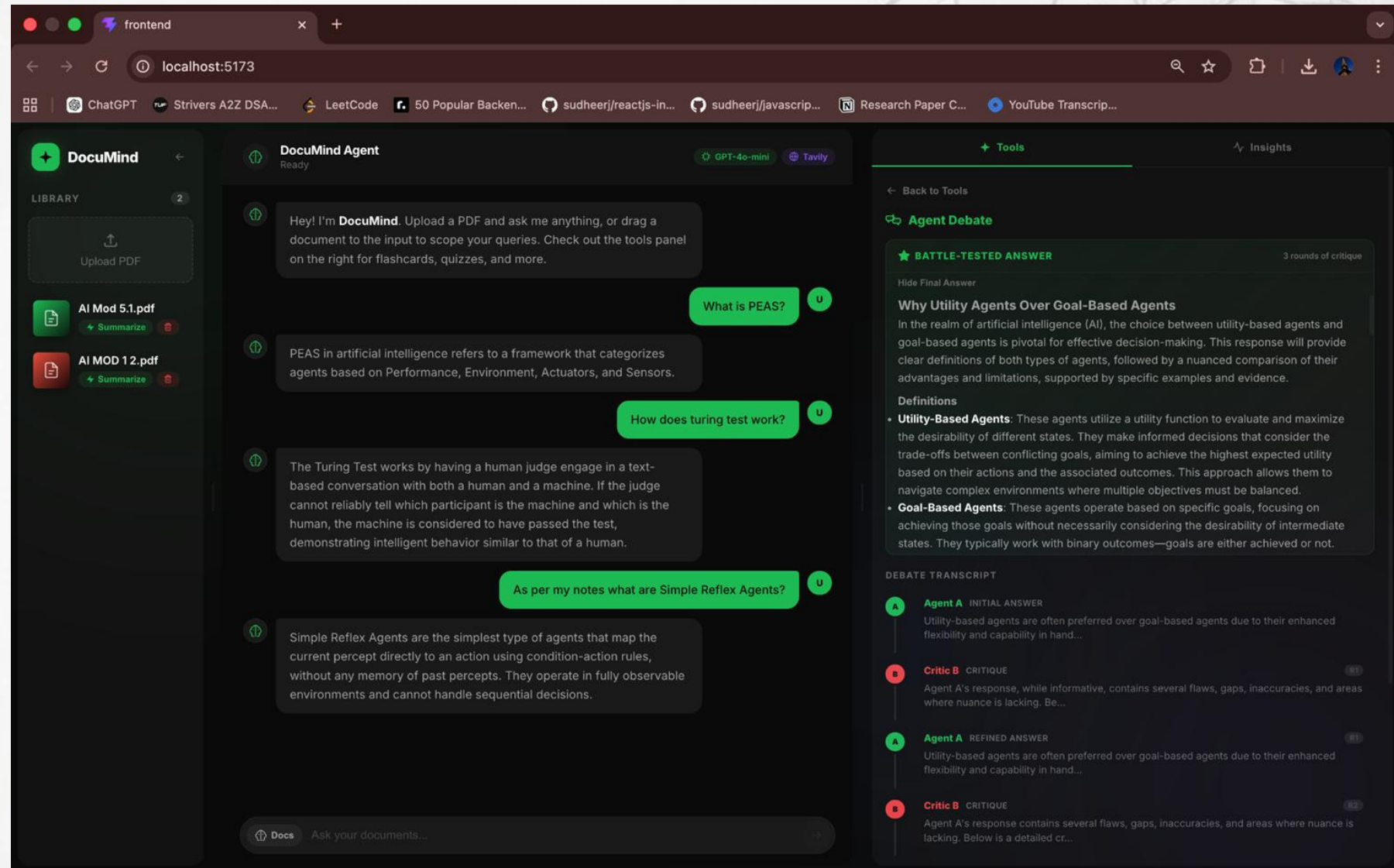
Flashcards Feature

- Generates Q&A pairs for revision
- Supports active recall learning



Agent Debate

- Simulates multi-agent discussion
- Enhances critical understanding of concepts

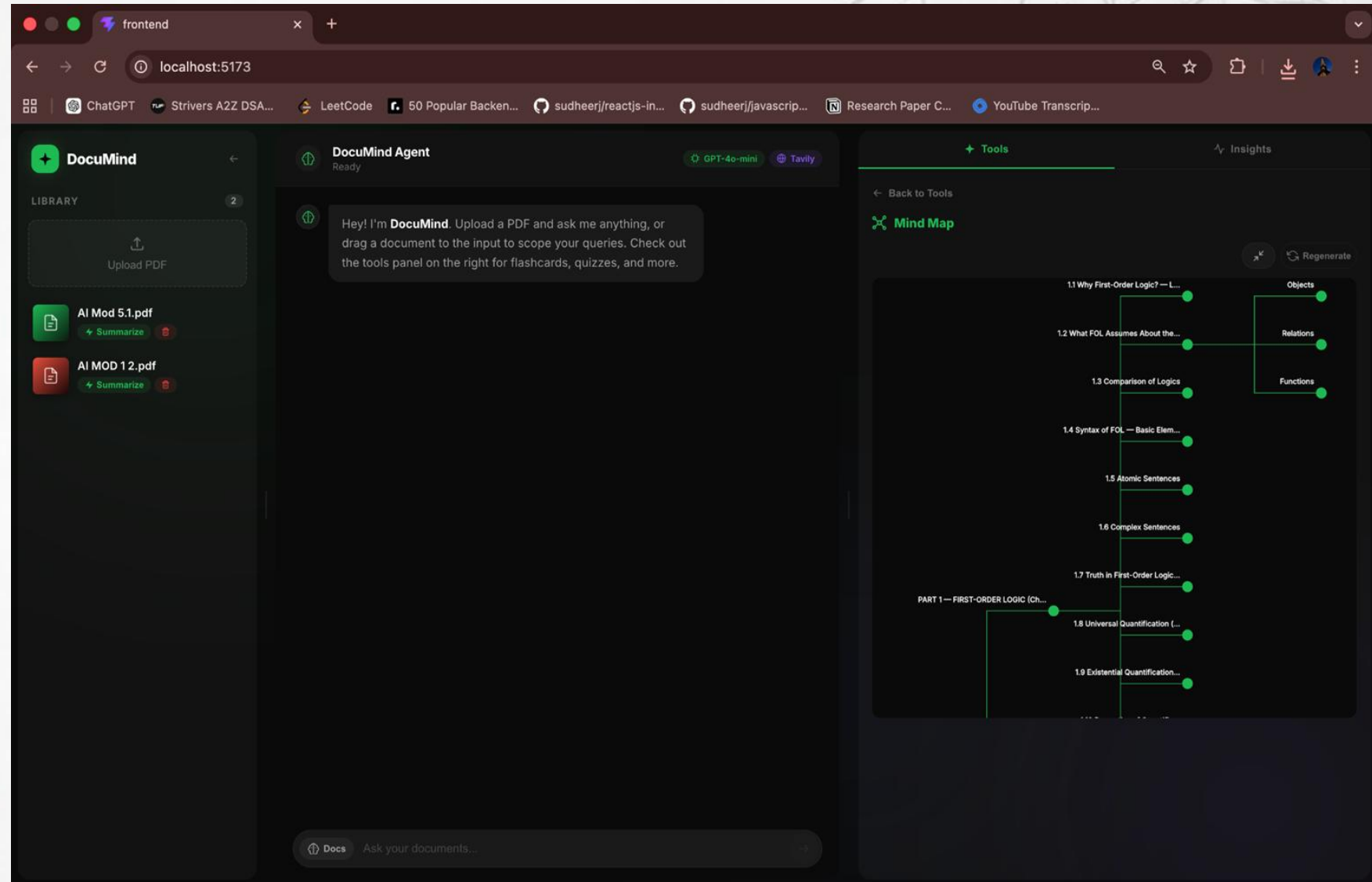


The screenshot displays the DocuMind Agent interface in a web browser. The interface is divided into several sections:

- Library:** A sidebar on the left showing two PDF documents: "AI Mod 5.1.pdf" and "AI MOD 12.pdf", each with a "Summarize" button.
- DocuMind Agent:** The main chat area shows a conversation with the agent. The agent's initial message is: "Hey! I'm **DocuMind**. Upload a PDF and ask me anything, or drag a document to the input to scope your queries. Check out the tools panel on the right for flashcards, quizzes, and more." The user asks: "What is PEAS?". The agent responds: "PEAS in artificial intelligence refers to a framework that categorizes agents based on Performance, Environment, Actuators, and Sensors." The user asks: "How does turing test work?". The agent responds: "The Turing Test works by having a human judge engage in a text-based conversation with both a human and a machine. If the judge cannot reliably tell which participant is the machine and which is the human, the machine is considered to have passed the test, demonstrating intelligent behavior similar to that of a human." The user asks: "As per my notes what are Simple Reflex Agents?". The agent responds: "Simple Reflex Agents are the simplest type of agents that map the current percept directly to an action using condition-action rules, without any memory of past percepts. They operate in fully observable environments and cannot handle sequential decisions."
- Tools:** A panel on the right titled "Tools" with a sub-section "Agent Debate". It displays a "BATTLE-TESTED ANSWER" for the question "Why Utility Agents Over Goal-Based Agents". The answer includes a definition of utility-based agents and a comparison with goal-based agents. It also lists "Definitions" for "Utility-Based Agents" and "Goal-Based Agents".
- DEBATE TRANSCRIPT:** A section at the bottom right showing the transcript of the debate. It includes the initial answer from Agent A and the critique from Critic B, followed by a refined answer from Agent A and another critique from Critic B.

Mind Map

- Visualizes concept relationships
- Helps in structured learning and navigation



The screenshot displays the DocuMind application interface. On the left, a 'LIBRARY' panel shows two PDF documents: 'AI Mod 5.1.pdf' and 'AI MOD 1 2.pdf', each with a 'Summarize' button. The central area features a chat window with the 'DocuMind Agent' (GPT-4o-mini) and a prompt: 'Hey! I'm DocuMind. Upload a PDF and ask me anything, or drag a document to the input to scope your queries. Check out the tools panel on the right for flashcards, quizzes, and more.' On the right, the 'Tools' panel is active, showing a 'Mind Map' tool. The mind map visualizes the structure of the document, with a central node 'PART 1 — FIRST-ORDER LOGIC (Ch...)' branching into '1.1 Why First-Order Logic? — L...', '1.2 What FOL Assumes About the...', '1.3 Comparison of Logic', '1.4 Syntax of FOL — Basic Elem...', '1.5 Atomic Sentences', '1.6 Complex Sentences', '1.7 Truth in First-Order Logic...', '1.8 Universal Quantification (...)', and '1.9 Existential Quantification...'. A sidebar on the right lists 'Objects', 'Relations', and 'Functions'.

Classical vs Modern Agent

Feature	Classical Agent	DocuMind (Modern LLM Agent)
Decision Logic	Predefined rules (if-else, condition-action)	LLM-driven reasoning based on prompt understanding
Planning Strategy	Static, fixed execution path	Dynamic planning using ReAct (Reason + Act)
Memory	No memory or very limited state	Short-term contextual memory (chat history)
Execution Style	Single-step response	Multi-step reasoning with iterative refinement
Tool Usage	Hardcoded tool invocation	Autonomous tool selection via descriptions
Knowledge Source	Fixed knowledge base	Hybrid: FAISS (documents) + Web search
Adaptability	Cannot handle unseen queries well	Generalizes to new queries using LLM reasoning
Transparency	No visibility into decision process	Observable Thought → Action → Observation logs
Scalability	Difficult to extend (needs new rules)	Easily extensible by adding new tools